

Common Part 1: Polynomial Graph Algorithms, Combinatorics, Search Trees, and Automata

Common Part 1: PAL

Polynomial algorithms for standard graph problems. Combinatorial and number-theoretical algorithms, isomorphism, prime numbers. Search trees and their use. Text search based on finite automata.

Primary source repositories consulted: mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard algorithmic definitions.

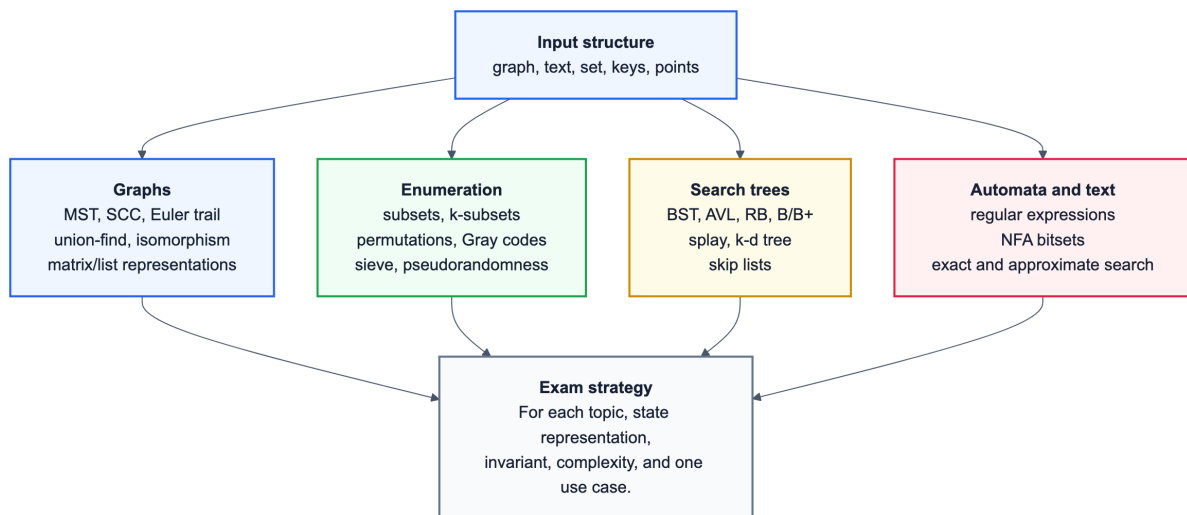


Figure 1: PAL topic map: graph algorithms, enumeration, search trees, and automata/text search.

PAL in Plain English

PAL is not one big idea. It is five small toolboxes. Learn what each toolbox is for, the main names, and one sentence you can say in the exam.

1. Recognize the topic.

Graph, generation, tree, or automaton?

2. Say the landmark.

Condition, algorithm name, or formula.

3. Finish with complexity.

Most PAL answers end with $O(\dots)$.

1

Graph basics

How to store a graph and talk about its size.

Know this

- **$n = \text{vertices}$, $m = \text{edges}$.**
- Adjacency list: good for sparse graphs, $O(n + m)$ memory.
- Adjacency matrix: $O(1)$ edge lookup, but $O(n^2)$ memory.

Watch out

- Distance matrix usually means shortest-path distances.
- Laplacian/incidence matrices are the algebraic view.

Exam sentence: For sparse graphs I would use adjacency lists because they take $O(n + m)$ memory; matrices are useful when I need $O(1)$ edge queries.

2

Graph algorithms

MST, components, Euler trails, and isomorphism.

Core algorithms

- **Prim/Kruskal/Boruvka** build minimum spanning trees.
- **Union-find** tracks components for Kruskal.
- **Kosaraju/Tarjan** find SCCs in $O(n + m)$.

Key conditions

- MST: cut property and cycle property.
- Euler undirected: all even, or exactly two odd.
- Euler directed: in-degree/out-degree balance.

Exam sentence: Kruskal sorts edges and adds an edge only if union-find says it connects two different components; sorting gives $O(m \log n)$.

3

Generation and numbers

Listing objects, finding primes, and simple pseudorandomness.

Counts

- Subsets: 2^n .
- k -subsets: $C(n, k)$.
- Permutations: $n!$.

Names to remember

- Gray code: consecutive bit strings differ in one bit.
- Sieve of Eratosthenes: all primes up to N .
- LCG: $X' = (aX + c) \bmod m$.

Exam sentence: The sieve marks multiples starting from p^2 and finds all primes up to N in $O(N \log \log N)$ time.

PAL in Plain English

Second half: search trees and automata/text search. These are mostly "choose the right structure" and "remember how the pattern is tracked."

4

Search trees

Different dictionaries for different workloads.

Balanced trees

- AVL: stricter balance, $O(\log n)$.
- Red-black: also $O(\log n)$, common in libraries.
- Splay: amortized $O(\log n)$, good with repeated access.

Special trees

- B/B+ trees: block storage and database indexes.
- k-d tree: geometric points in low dimensions.
- Skip list: randomized ordered dictionary.

Exam sentence: All balanced search trees aim for logarithmic height; the difference is how they maintain balance and what workload they are optimized for.

5

Automata and text search

A finite state machine remembers how much of the pattern has matched.

Automata

- DFA: exactly one next state.
- NFA: a set of possible states.
- Regex, DFA, and NFA describe regular languages.

Text search

- KMP: exact matching in $O(n + m)$.
- Levenshtein DP: approximate matching in $O(mn)$.
- Aho-Corasick: many patterns in one scan.

Exam sentence: KMP can be seen as an automaton over prefix lengths: after a mismatch, failure links say how much of the prefix is still usable.

How to study PAL in two days

First pass

Read these two pages and say each exam sentence out loud. You only need the rough map first.

Second pass

Go into the notes and attach details to each box: definitions, algorithms, and complexity.

Final pass

Cover the notes and answer from memory: "what is it, what algorithm, what complexity, what trap?"

Three common traps

- Do not mix directed and undirected Euler conditions.
- Graph invariants can disprove isomorphism, but usually do not prove it.
- $O(1)$ matrix edge lookup is bought with $O(n^2)$ memory.

1. Asymptotic Complexity and Graph Notation

Asymptotic notation

For non-negative functions $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

- $f(n) \in O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.
- $f(n) \in \Omega(g(n))$ iff $g(n) \in O(f(n))$.
- $f(n) \in \Theta(g(n))$ iff $f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$.
- $f(n) \in o(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- $f(n) \in \omega(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Typical growth order:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

For graph algorithms, let $n = |V|$ and $m = |E|$.

Simple explanation

Asymptotic notation is a way to ignore machine speed and tiny implementation details. If one algorithm is $O(n)$ and another is $O(n^2)$, the first one scales much better: doubling the input roughly doubles its work, while the second one roughly quadruples it.

Basic graph terms

A graph is usually $G = (V, E)$. In an undirected graph, edges are unordered pairs $\{u, v\}$; in a directed graph, arcs are ordered pairs (u, v) .

- **Degree:** in an undirected graph, $\deg(v)$ is the number of incident edges. In directed graphs use $\deg^+(v)$ for out-degree and $\deg^-(v)$ for in-degree.
- **Walk:** sequence $v_0, e_1, v_1, \dots, e_k, v_k$ where each edge joins consecutive vertices.
- **Trail:** a walk with no repeated edge.
- **Path:** a walk with no repeated vertex.
- **Circuit / closed trail:** a trail that starts and ends in the same vertex.
- **Cycle:** a closed path, usually with at least one edge and no repeated vertices except the first/last.
- **Connected component:** maximal set of mutually reachable vertices in an undirected graph.
- **Strongly connected component:** maximal set of vertices in a directed graph where every vertex is reachable from every other.
- **Tree:** connected undirected graph with no cycles; equivalently, $|E| = |V| - 1$ and connected.

Graph representations

Adjacency matrix. Matrix $A \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ iff edge (i, j) exists. For weighted graphs, store weights or ∞ for missing edges.

- Memory: $O(n^2)$.
- Edge existence query: $O(1)$.
- Iterating neighbors of one vertex: $O(n)$.
- Good for dense graphs.

Distance matrix. Matrix D , where D_{ij} stores the shortest-path distance from i to j , or ∞ if unreachable. It is not merely an input representation; it is often the output of all-pairs shortest paths.

Laplacian matrix. For an undirected graph,

$$L = \Delta - A,$$

where Δ is the diagonal matrix of degrees. L is central in spectral graph theory. The number of spanning trees can be obtained from any cofactor of L by Kirchhoff's matrix-tree theorem.

Incidence matrix. Matrix $B \in \{-1, 0, 1\}^{n \times m}$ for directed graphs: column $e = (u, v)$ has -1 at u , $+1$ at v , and zero elsewhere. For undirected graphs, entries are usually 1 at both endpoints.

Adjacency list. For each vertex, store a list of its outgoing neighbors.

- Memory: $O(n + m)$.
- Iterating all edges: $O(n + m)$.
- Edge existence query: $O(\deg(v))$, or expected $O(1)$ if neighbor sets are hashed.
- Best default for sparse graphs.

2. Minimum Spanning Trees, SCCs, Euler Trails, Union-Find, Isomorphism

Minimum spanning tree

Given connected undirected weighted graph $G = (V, E, w)$, a spanning tree is a connected acyclic subgraph containing all vertices. A minimum spanning tree (MST) has minimum total weight.

Important correctness principles:

- **Cut property:** for any cut, a lightest edge crossing the cut is safe for some MST.
- **Cycle property:** for any cycle, a heaviest edge on the cycle is not needed in some MST.

If edge weights are distinct, the MST is unique.

Prim-Jarnik algorithm

Prim grows one tree from an arbitrary start vertex.

1. Put the start vertex in the tree.
2. Repeatedly add the minimum-weight edge crossing from the tree to an outside vertex.
3. Stop when all vertices are included.

With an adjacency list and binary heap: $O(m \log n)$. With a Fibonacci heap: $O(m + n \log n)$.

With an adjacency matrix: $O(n^2)$, which is good for dense graphs.

Kruskal algorithm

Kruskal grows a forest.

1. Sort all edges by nondecreasing weight.
2. Scan edges in that order.
3. Add an edge iff it connects two different components.
4. Use union-find to maintain components.

Complexity: $O(m \log m)$ for sorting, i.e. $O(m \log n)$, plus almost-linear union-find overhead.

Example

Suppose edges sorted by weight are $(A, B) : 1$, $(B, C) : 2$, $(A, C) : 3$, $(C, D) : 4$. Kruskal first takes (A, B) , then (B, C) . It skips (A, C) because A and C are already connected, so adding it would create a cycle. Then it takes (C, D) . The MST edges are $(A, B), (B, C), (C, D)$.

Boruvka algorithm

Boruvka repeatedly connects every component using its cheapest outgoing edge.

1. Initially every vertex is one component.
2. For each component, find its lightest outgoing edge.
3. Add all such edges simultaneously and merge components.
4. Repeat until one component remains.

Each phase at least halves the number of components, so there are $O(\log n)$ phases. A simple implementation runs in $O(m \log n)$.

Union-find

Union-find maintains a partition of elements into disjoint sets.

- **make-set(x)**: create singleton set.
- **find(x)**: return representative of the set containing x .
- **union(x,y)**: merge the sets containing x and y .

Efficient implementation:

- Store each set as a rooted tree.
- Use **union by rank/size**: attach the shallower/smaller tree below the deeper/larger one.
- Use **path compression** during **find**: make visited nodes point directly to the root.

For q operations on n elements, complexity is $O((n+q)\alpha(n))$, where α is the inverse Ackermann function, effectively constant in practice.

Simple explanation

Union-find is useful when the only question is “are these two things already in the same group?” In Kruskal’s MST algorithm, each accepted edge merges two groups. Path compression makes future questions faster by making every visited vertex point almost directly to the group representative.

Strongly connected components

A strongly connected component (SCC) in a directed graph is a maximal subset $C \subseteq V$ such that for all $u, v \in C$, both $u \rightsquigarrow v$ and $v \rightsquigarrow u$.

The condensation graph of SCCs is always a DAG.

Kosaraju-Sharir algorithm

1. Run DFS on G and record vertices in decreasing finish time.
2. Reverse all edges to get G^R .
3. Process vertices in decreasing finish time; each DFS tree in G^R is one SCC.

Complexity: $O(n + m)$. It is simple, but needs the reversed graph and two DFS passes.

Tarjan algorithm Tarjan finds SCCs in one DFS using a stack.

For each vertex v maintain:

- **index[v]**: DFS discovery order.
- **lowlink[v]**: smallest discovery index reachable from v using zero or more tree edges plus at most one back edge to a vertex still on the stack.

When **lowlink[v] == index[v]**, v is the root of an SCC; pop stack vertices until v is popped.

Complexity: $O(n + m)$.

Euler trail

An Euler trail uses every edge exactly once. An Euler circuit is a closed Euler trail.

For an undirected graph, ignoring isolated vertices:

- Euler circuit exists iff every vertex has even degree and the non-isolated part is connected.
- Euler trail with different endpoints exists iff exactly two vertices have odd degree.

For a directed graph, ignoring isolated vertices and requiring all nonzero-degree vertices to lie in one weakly connected component:

- Euler circuit exists iff $\deg^+(v) = \deg^-(v)$ for every vertex.
- Euler trail from s to t exists iff $\deg^+(s) = \deg^-(s) + 1$, $\deg^-(t) = \deg^+(t) + 1$, and all other vertices have equal in/out degree.

Hierholzer algorithm: follow unused edges until a circuit/trail closes, then splice in additional circuits where unused edges remain. Runs in $O(n + m)$.

Exam tip

Do not mix the undirected and directed Euler conditions. Undirected graphs use even/odd degree. Directed graphs use equality of in-degree and out-degree, except possibly at the start and end vertices of an open trail.

Graph isomorphism

Graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if there is a bijection $f : V_1 \rightarrow V_2$ such that

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

Necessary invariants include equal numbers of vertices and edges, degree multiset, connected-component sizes, cycle counts, spectrum of the adjacency/Laplacian matrix, etc. These invariants can quickly disprove isomorphism, but most are not complete: equal invariant values do not always imply isomorphism.

General graph isomorphism is in NP. It is not known to be in P nor NP-complete; the best-known general theoretical result is quasipolynomial time. Practical solvers use refinement, canonical labeling, and backtracking.

Tree isomorphism

Tree isomorphism is solvable in linear or near-linear time.

For unrooted trees:

1. Find the center(s) by repeatedly removing leaves, or via longest path midpoint.
2. Root each tree at its center. If a tree has two centers, try both rootings.
3. Compute a canonical code bottom-up: a leaf has code $()$, and an internal node has code consisting of sorted child codes inside parentheses.
4. Trees are isomorphic iff their canonical root codes match.

Sorting child codes naively gives $O(n \log n)$; careful integer relabeling by levels gives $O(n)$.

3. Combinatorial Generation, Prime Numbers, and Pseudorandom Numbers

Subsets

All subsets of an n -element set can be generated by bit masks from 0 to $2^n - 1$. Bit i tells whether element i is included.

- Count: 2^n .
- Time to output all subsets: $\Theta(n2^n)$ if each subset is printed explicitly.

Recursive include/exclude generation:

```
generate(i):
    if i == n: output current subset
    else:
        exclude element i; generate(i+1)
        include element i; generate(i+1)
```

k-element subsets

There are $\binom{n}{k}$ k-element subsets. Common generation methods:

- Recursive choice with pruning.
- Lexicographic next-combination algorithm.
- Revolving-door order, where consecutive combinations differ by one removed and one inserted element.

Lexicographic next-combination for indices $a_1 < \dots < a_k$:

1. Find largest i such that $a_i < n - k + i$.
2. Increment a_i .
3. Set $a_j = a_{j-1} + 1$ for all $j > i$.

Permutations

There are $n!$ permutations.

Useful algorithms:

- Lexicographic **next_permutation**: find longest non-increasing suffix, swap pivot with the next larger element in suffix, reverse suffix.
- Heap's algorithm: simple recursive generation with swaps.
- Johnson-Trotter: adjacent transpositions; consecutive permutations differ by swapping neighboring elements.

Gray codes

A binary Gray code lists all 2^n bit strings so that consecutive strings differ in exactly one bit. The reflected binary Gray code can be computed as

$$G(i) = i \oplus (i \gg 1).$$

Gray codes are useful when changing one decision at a time is cheaper than rebuilding a full object, e.g. iterating subsets while maintaining an aggregate value.

Example

For $n = 3$, reflected Gray code is 000, 001, 011, 010, 110, 111, 101, 100. Notice that each neighboring pair differs in one bit only. This is useful when each bit means “include this item in the subset” and you want to update the subset by one small change.

Prime numbers and sieve of Eratosthenes

A prime number is an integer $p > 1$ with only divisors 1 and p .

Trial division tests primality of n by trying divisors up to $\sqrt{n}$, so it takes $O(\sqrt{n})$ arithmetic divisions.

The sieve of Eratosthenes finds all primes $\leq N$:

1. Mark all numbers $2, \dots, N$ as potential primes.
2. For $p = 2, 3, \dots, \lfloor \sqrt{N} \rfloor$, if p is still marked, mark all multiples $p^2, p^2 + p, \dots$ composite.
3. Remaining marked numbers are primes.

Complexity: $O(N \log \log N)$ time and $O(N)$ memory.

For very large numbers, probabilistic primality tests such as Miller-Rabin are common; deterministic polynomial-time primality testing exists, but is less practical for ordinary use.

Pseudorandom numbers and LCG

A pseudorandom number generator (PRNG) is deterministic but should imitate randomness for intended statistical purposes.

Important properties:

- Long period.
- Uniform distribution over the output range.
- Low correlation between consecutive values.
- Reproducibility from a seed.
- For cryptography, unpredictability is also required; ordinary LCGs are not cryptographically secure.

A linear congruential generator (LCG) is

$$X_{n+1} = (aX_n + c) \bmod m.$$

The Hull-Dobell theorem gives full period m for a mixed LCG iff:

1. $\gcd(c, m) = 1$.
2. Every prime factor of m divides $a - 1$.
3. If $4 \mid m$, then $4 \mid a - 1$.

4. Search Trees

Binary search tree

A binary search tree (BST) stores keys with invariant:

$$\text{left subtree keys} < \text{node key} < \text{right subtree keys}.$$

Operations search/insert/delete take $O(h)$ where h is tree height. In a balanced tree $h = O(\log n)$; in the worst unbalanced case $h = O(n)$.

AVL tree

An AVL tree is a self-balancing BST where every node has balance factor

$$bf(v) = \text{height}(\text{left}(v)) - \text{height}(\text{right}(v)) \in \{-1, 0, 1\}.$$

After insertion/deletion, rotations restore balance:

- single right rotation,
- single left rotation,
- left-right rotation,
- right-left rotation.

Height is $O(\log n)$, so search, insert, and delete are $O(\log n)$.

Red-black tree

A red-black tree is a BST with color bits satisfying:

1. Every node is red or black.
2. The root is black.
3. Leaves/NIL sentinels are black.
4. A red node has black children.
5. Every path from a node to descendant NIL leaves has the same number of black nodes.

These rules imply height at most $2 \log_2(n+1)$. Search/insert/delete are $O(\log n)$. Red-black trees are often slightly less strictly balanced than AVL trees, but need fewer rotations in update-heavy workloads.

B-tree

A B-tree is a balanced multiway search tree optimized for block storage.

For minimum degree t :

- Each internal node except root has between $t - 1$ and $2t - 1$ keys.
- Each internal node with k keys has $k + 1$ children.
- All leaves are at the same depth.

Operations are $O(\log_t n)$ node visits. Since one node can match a disk/cache block, B-trees are used heavily in databases and filesystems.

B+ tree

A B+ tree stores all records in leaves; internal nodes contain only separator keys. Leaves are linked in sorted order.

Advantages over B-trees:

- Better range scans.
- Higher fanout in internal nodes.
- Stable leaf-level sequential access.

This is the standard index structure in many database systems.

Splay tree

A splay tree is a self-adjusting BST. After every access, insertion, or deletion helper access, the touched node is moved to the root by rotations:

- **zig**: parent is root.
- **zig-zig**: node and parent are both left children or both right children.
- **zig-zag**: node and parent are opposite-side children.

Worst-case single operation can be $O(n)$, but amortized complexity is $O(\log n)$. Splay trees exploit temporal locality: recently accessed keys become fast to access again.

k-d tree

A k-d tree stores points in k -dimensional space. At depth d , split by coordinate $d \bmod k$.

Construction:

1. Choose splitting coordinate.
2. Choose median point by that coordinate.
3. Recurse on points below/above the median.

Balanced construction costs $O(n \log n)$ or better with selection algorithms. Orthogonal range queries and nearest-neighbor queries are efficient in low dimensions, but performance deteriorates in high dimensions.

Nearest neighbor search in a k-d tree

1. Descend to the leaf where the query point would be inserted.
2. Keep current best point and distance.
3. Backtrack. At each split, first explore the side containing the query.
4. Explore the other side only if its bounding region could contain a closer point than the current best.

Average performance is often $O(\log n)$ in low dimensions, but worst case is $O(n)$.

Skip list

A skip list is a probabilistic ordered dictionary with multiple linked-list levels. Each element is promoted to the next level with probability p , commonly $1/2$.

Expected height is $O(\log n)$. Search, insert, and delete take expected $O(\log n)$ time and $O(n)$ expected space. Skip lists are simpler than balanced trees and work well in concurrent settings.

Complexity summary

Structure	Search	Insert	Delete	Notes
Unbalanced BST	$O(h)$, worst $O(n)$	$O(h)$	$O(h)$	Simple, input-order sensitive
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Strict balance
Red-black	$O(\log n)$	$O(\log n)$	$O(\log n)$	Common library map/set
B-tree / B+ tree	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	Block-oriented storage
Splay	amortized $O(\log n)$	amortized $O(\log n)$	amortized $O(\log n)$	Adapts to access locality
k-d tree	avg. $O(\log n)$	avg. $O(\log n)$	varies	Geometric data
Skip list	expected $O(\log n)$	expected $O(\log n)$	expected $O(\log n)$	Randomized

5. Finite Automata, Regular Languages, and Text Search

Finite automata

A deterministic finite automaton (DFA) is

$$A = (Q, \Sigma, \delta, q_0, F),$$

where Q is a finite state set, Σ alphabet, $\delta : Q \times \Sigma \rightarrow Q$ transition function, q_0 initial state, and $F \subseteq Q$ accepting states.

A nondeterministic finite automaton (NFA) allows multiple transitions and possibly ϵ -transitions:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q.$$

DFA and NFA recognize exactly the regular languages. Every NFA can be determinized by subset construction, possibly producing up to $2^{|Q|}$ DFA states.

Regular expressions and regular operations

Regular expressions denote regular languages. Basic constructs:

- $\emptyset$, ϵ , symbols $a \in \Sigma$.
- Union $R|S$.
- Concatenation RS .
- Kleene star R^* .

Regular languages are closed under union, intersection, complement, concatenation, Kleene star, difference, reversal, and homomorphism. Closure is usually proved by automaton constructions, e.g. product automata for intersection.

Bit representation of NFAs

For small/medium automata, the active NFA state set can be stored as a bit vector. One transition step becomes bit operations:

1. Maintain bitset S of active states.
2. For input character a , compute next active states using precomputed transition masks.
3. Apply epsilon closure if needed.

This is the basis of bit-parallel text searching. On a word-RAM with word size w , simulating m pattern states can cost $O(\lceil m/w \rceil)$ per text character.

Simple explanation

An NFA can be in many states at once. A bit vector is just a compact checklist of those active states. Instead of looping over states one by one, the computer can update many bits with one CPU operation.

Exact pattern matching

Given pattern P of length m and text T of length n .

Naive search: try every alignment and compare characters. Worst-case $O(nm)$.

Automaton/KMP view: build a DFA-like prefix automaton whose state is the length of the longest prefix of P matched so far. KMP computes failure links/prefix function in $O(m)$ and scans in $O(n)$, total $O(n + m)$.

Boyer-Moore family: compares from the end of the pattern and uses bad-character/good-suffix shifts. Worst-case variants can be linear; in practice it is often sublinear on natural text.

Shift-And / Shift-Or: bit-parallel exact matching. Precompute character masks for pattern positions. Each text character updates a bit vector; a match is found when the accepting bit is set. Complexity $O(n \lceil m/w \rceil)$.

Approximate pattern matching

Hamming distance between equal-length strings is the number of positions where they differ. Approximate matching with Hamming distance at most k allows substitutions only.

Levenshtein distance is edit distance with insertions, deletions, and substitutions. Dynamic programming recurrence:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{delete} \\ D[i, j-1] + 1 & \text{insert} \\ D[i-1, j-1] + [P_i \neq T_j] & \text{substitute/match} \end{cases}$$

Full DP is $O(mn)$. If only distance $\leq k$ matters, compute a diagonal band of width $2k + 1$ for $O(kn)$. Bit-parallel algorithms such as Myers' algorithm can be much faster in practice for moderate pattern length.

Example

The Levenshtein distance between **cat** and **cut** is 1: substitute **a** by **u**. The distance between **cat** and **cart** is also 1: insert **r**. Hamming distance would not allow insertion, so it only applies to equal-length strings.

Approximate matching can also be modeled by an automaton whose states encode pattern position and number of errors used. For Levenshtein distance k , the automaton has roughly $O(mk)$ relevant states.

Dictionary automata

For a set of patterns, a trie recognizes prefixes of dictionary words. Aho-Corasick augments the trie with failure links, analogous to KMP failure links, so all dictionary occurrences in a text are found in one pass.

Construction:

1. Insert all patterns into a trie.
2. Build failure links by BFS.
3. Store output lists at terminal states and inherited failure states.
4. Scan text by transitions plus failure fallback.

Complexity:

- Construction: $O(\text{total pattern length} \cdot |\Sigma|)$ for full transition tables, or linear in stored transitions with maps.
- Search: $O(n + z)$, where z is the number of reported occurrences.

Exam checklist

Before answering, be able to:

- Define graph representations and compare their complexities.
- State MST algorithms, their invariants, and their time complexity.
- Distinguish undirected and directed Euler conditions.
- Explain union-find optimizations and the inverse Ackermann bound.
- Explain why tree isomorphism is easy but general graph isomorphism is special.
- Generate subsets, k -subsets, permutations, and Gray codes.
- Give sieve and LCG formulas and complexities.
- Compare AVL, red-black, B/B+, splay, k -d trees, and skip lists.
- Define DFA/NFA/regular expressions and connect automata to exact, approximate, and dictionary text search.